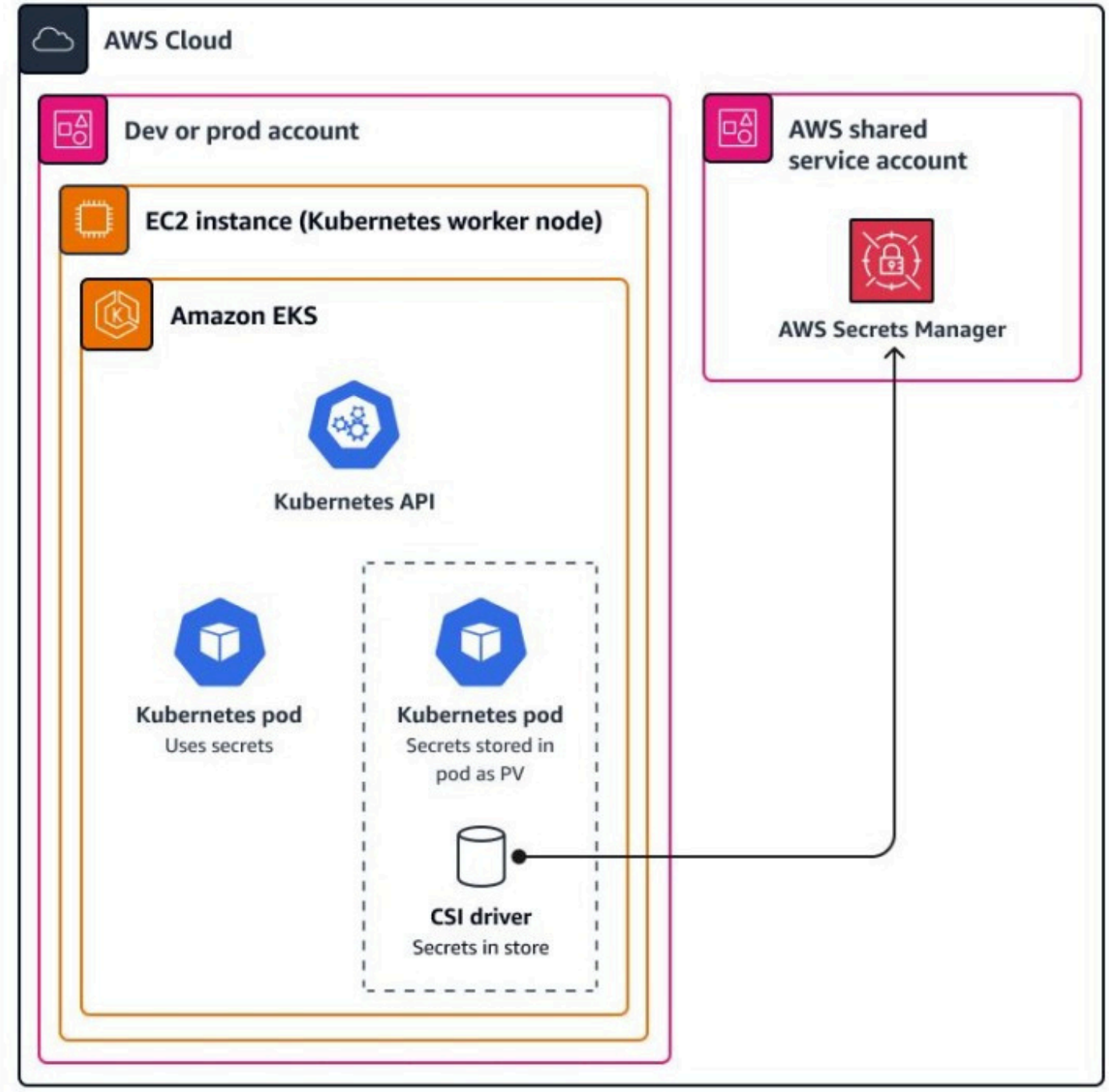

Use Case to access AWS secrets from pods in EKS Cluster

Situation: You have an application running in pods within an EKS cluster that requires sensitive information like database passwords, API keys, or certificates. You want to securely manage and provide access to these secrets to your pods without hardcoding them into your application code or configurations.

Task: Attach AWS Secrets Manager secrets to your pods running in the EKS cluster securely.

Action:

Create Secrets in AWS Secrets Manager:

- Log in to the AWS Management Console.

- Navigate to AWS Secrets Manager.
- Create a new secret for each sensitive piece of information your application requires (e.g., database password, API key).
- Store the sensitive information securely within these secrets.

Set Up IAM Role for Pods:

- Create an IAM role that allows your pods to access the secrets stored in AWS Secrets Manager.
- Attach a policy to this IAM role that grants necessary permissions to access the secrets

Install Kubernetes CSI Driver for AWS Secrets Manager:

- To enable Kubernetes to interact with AWS Secrets Manager, you need to install the Kubernetes CSI driver for AWS Secrets Manager.
- This driver enables Kubernetes to mount secrets stored in AWS Secrets Manager as volumes in pods.

Modify Kubernetes Manifests:

- Update your pod's Kubernetes manifest file to include a volume mount for the AWS Secrets Manager secret.
- Define a volume in the pod spec that references the AWS Secrets Manager secret using the CSI driver.
- Define a volume mount in the container spec to mount the volume at the desired path within the container.

Configure Pod to Use Secrets:

- Within your container, read the sensitive information from the mounted volume at the specified path.
- Modify your application code or configuration files to use the sensitive information obtained from the mounted volume.

Result: Your pods running in the EKS cluster are securely accessing sensitive information stored in AWS Secrets Manager without exposing them in plaintext or hardcoding them into your application code or configurations.

Here's an example Kubernetes manifest snippet demonstrating how to mount an AWS Secrets Manager secret into a pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: my-app-container
      image: my-app-image:latest
      volumeMounts:
        - name: secret-volume
          mountPath: /var/my-app/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      csi:
        driver: secrets-store.csi.k8s.io
        readOnly: true
        volumeAttributes:
          secretProviderClass: "aws-secretsmanager-secret-provider"
```

Ensure that you replace my-app-image:latest with your application image and adjust the mountPath to match your application's requirements. Also, make sure to define the secretProviderClass attribute appropriately based on your CSI driver configuration.

By following these steps, you can securely attach AWS Secrets Manager secrets to pods in your EKS cluster using Kubernetes manifests and Kubernetes CSI addons for AWS.

How to configure addons and CSI drivers :

To install and configure the necessary addons and CSI drivers for using AWS Secrets Manager with Kubernetes pods in an EKS cluster, follow these steps:

Install AWS IAM Authenticator:

- Before installing any addons, ensure you have the AWS IAM Authenticator installed and configured in your environment. This allows Kubernetes to authenticate against your EKS cluster using IAM credentials.
- You can download the AWS IAM Authenticator from the GitHub releases page
- Follow the installation instructions provided for your operating system.

Install AWS CLI:

- Make sure you have the AWS CLI installed and configured with appropriate IAM credentials that have permissions to manage your EKS cluster and resources.
- You can download and install the AWS CLI from the official AWS documentation: Installing the AWS CLI.

Install EKS CSI Driver:

- The AWS EBS (Elastic Block Store) CSI driver is required for dynamic provisioning of EBS volumes and can be used alongside the AWS Secrets Manager CSI driver.
- You can deploy the EKS CSI driver using the following command:kubectl apply -k "github.com/kubernetes-sigs/aws-ebs-csi-driver/tree/release-1.9/deploy/kubernetes/overlays/stable/ecr/?ref=release-1.9"

Install Secrets Store CSI Driver for AWS Secrets Manager:

- The Secrets Store CSI driver allows Kubernetes to mount secrets stored in AWS Secrets Manager as volumes in pods.
- Clone the Secrets Store CSI driver GitHub repository: git clone https://github.com/kubernetes-sigs/secrets-store-csi-driver.git
- Navigate to the secrets-store-csi-driver/deploy/kubernetes/ directory.
- Run the following command to deploy the AWS Secrets Manager CSI: *kubectl apply -f secrets-store-csi-driver/deploy/kubernetes/secretproviderclass-crd.yaml, kubectl apply -f secrets-store-csi-driver/deploy/kubernetes/provider-aws-secret-manager.yaml*

Create Secret Provider Class:

- Define a SecretProviderClass object that specifies the CSI driver to use and any additional configurations.
- An example SecretProviderClass for AWS Secrets Manager:

```yaml
apiVersion: secrets-store.csi.x-k8s.io/v1alpha1
kind: SecretProviderClass
metadata:
  name: aws-secretsmanager-secret-provider
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: <SECRET_NAME>
        objectType: secret
        objectVersion: <SECRET_VERSION>
  secretObjects:
    - secretName: <K8S_SECRET_NAME>
      type: Opaque
      data:
        - objectName: <SECRET_NAME>
          key: <SECRET_KEY>
```

- Replace <SECRET_NAME>, <SECRET_VERSION>, <K8S_SECRET_NAME>, and <SECRET_KEY> with appropriate values.

Deploy Pods:

- Update your pod manifests to reference the SecretProviderClass created in the previous step and mount the AWS Secrets Manager secret as a volume in your pods.
- Ensure that the pod's service account has the necessary IAM permissions to access the secrets stored in AWS Secrets Manager.

After completing these steps, your Kubernetes pods in the EKS cluster should be able to securely access secrets stored in AWS Secrets Manager using the CSI driver and the Secrets Store CSI driver addon.